

Project Progress Report: Week 3 & Week 4

Project Title: CodeDNA – Biomorphic Code Visualization Engine

Task Domain: Project Setup & Basic Code Parsing

1. Project Setup (Week 3)

During Week 3, the complete development environment was established and the project structure was organized in preparation for implementation. All necessary tools were installed and configured to ensure a smooth development process.

1.1 Development Environment

The following environment was set up:

- Java Development Kit (JDK) 25 installed and configured as the default JDK
- IntelliJ IDEA Community Edition installed as the primary IDE
- Project created in IntelliJ named 'CodeDNA' with JDK 25 selected

1.2 Project Structure

The project package and class files were organized as follows:

- Package created: codedna (inside the src/ directory)
- Five Java class files created inside the codedna package: ASTParser, CreatureGenerator, SVGBuilder, HTMLExporter, and Main
- Two directories created at the project root: sample/ (for input Java files) and output/ (for generated HTML output)

Project Directory Structure:

Directory / File	Purpose
src/codedna/ASTParser.java	Parses Java source file and extracts metrics
src/codedna/CreatureGenerator.java	Maps code metrics to creature features
src/codedna/SVGBuilder.java	Generates animated SVG organism
src/codedna/HTMLExporter.java	Wraps SVG in HTML browser output
src/codedna/Main.java	Entry point, runs full pipeline
sample/	Stores input .java files for analysis
output/	Stores generated HTML output files

1.3 Sample Input File

A sample Java file named FirstDemo.java was created and placed inside the sample/ directory. This file contains multiple methods, loops, and conditional statements to serve as a meaningful test input for the parsing module.

2. Basic Code Parsing (Week 4)

Week 4 focused on implementing the core parsing functionality of the CodeDNA system through the ASTParser class. This module is responsible for reading any Java source file and extracting all relevant structural metrics used to generate the biological creature.

2.1 File Reading Mechanism

The file reading mechanism was implemented using Java's built-in java.nio.file.Files and java.io libraries — requiring zero external dependencies. The parser reads the file line by line and processes each line to extract code elements.

2.2 Code Elements Extracted

The ASTParser successfully detects and counts the following elements:

- Classes, interfaces, and enums — using regex pattern matching
- Method declarations — identified by access modifiers and return types
- Loops — for, while, and do-while loops
- Nested loops — tracked using brace depth analysis
- Conditional statements — if blocks
- Try-catch blocks
- Import statements
- Comment lines and blank lines
- Top keywords — most frequently used identifiers in the file

2.3 Metrics Computed

In addition to raw element counts, the parser computes two derived scores:

Metric	Description
Complexity Score (0–100)	Based on method count, loop count, if count, and nested loop depth
Health Score (0–100)	Based on comment ratio and structural cleanliness

Method Line Counts	Tracks how many lines each method contains
Top Keywords	Extracts the 6 most frequently used identifiers

2.4 Sample Output

When ASTParser was run against FirstDemo.java, the console produced the following results:

```
==== CodeDNA Parser Results ====
File       : FirstDemo.java
Total Lines : 30
Code Lines  : 22
Classes    : 1  [FirstDemo]
Methods    : 3  [main, printNumbers, checkGrades]
Loops      : 2
Nested Loops : 0
If Statements : 3
Complexity Score: 24/100
Health Score  : 60/100
=====
```

3. OOP Concepts Applied

The following Object-Oriented Programming concepts were demonstrated in the Week 3 & 4 implementation:

- Encapsulation — The CodeMetrics inner class holds all parsed data fields privately within ASTParser
- Abstraction — The Main class calls the parse() method without needing to know internal parsing logic
- Separation of Concern — ASTParser handles only parsing; other classes will handle visualization and output
- Static Methods — All utility methods in ASTParser are static, avoiding unnecessary object instantiation
- Collections — List, Map, and ArrayList are used to store method names, class names, and keyword frequencies

4. Development Summary

By the end of Week 4, the first two implementation phases of the CodeDNA project have been completed successfully. The development environment is fully configured, the project structure is organized according to the defined architecture, and the core parsing module is functional and tested.

The ASTParser is capable of reading any Java source file and producing a comprehensive set of structural metrics that will serve as the biological DNA of the creature in the upcoming visualization stages. The project is now ready to proceed into Week 5, which focuses on advanced parsing including nested structure detection and comment analysis.